Console Gaming Subscription Packages

A Relational Database Design, Implementation, and Performance Evaluation

Final Project Report

SRH University: Relational DBMS

Anirudh Gopishankar 100001238

August 2026

Abstract

This report documents the design, implementation, and evaluation of a relational database for a Console Gaming Subscription Platform, developed as the final project for the Relational DBMS course at SRH University. The system supports three user groups — gamers, payment/customer service agents, and (indirectly) game publishers — and is built around ten entities covering players, accounts, consoles, subscription tiers, games, publishers, billing transactions, and customer support. The central business goal is to prevent a customer from being overcharged for a game they already own or a subscription they already hold. The design was normalized to Third Normal Form (3NF), implemented as a MySQL 8.0 database with full constraint enforcement and realistic sample data, and queried using a set of advanced SQL techniques including joins, subqueries, window functions, and common table expressions. The report closes with a discussion of database performance at scale, ACID compliance, and the CAP theorem, each grounded in concrete examples from the project.

Table of Contents

Abstract 2
Table of Contents 3

Chapter 1: Introduction and Requirements Analysis 4
1.1 Scenario 4
1.2 Business Goals 4
1.3 Users 4
1.4 Data Requirements 4
1.5 Business Rules 5

Chapter 2: Database Design, ER Diagram, and Normalization 6
2.1 Entity List 6
2.2 Relationships — Cardinality and Participation 6
2.3 Entity:Relationship Diagram 7
2.4 Functional Dependencies 7
2.5 Normalization 8

Chapter 3: Database Implementation 10
3.1 Technology and Approach 10
3.2 Illustrative Table Definition 10
3.3 Sample Data 10
3.4 Verification 11-12

Chapter 4: SQL Query Analysis 13

Chapter 5: Database Performance Discussion 19
5.1 Factors Affecting Database Performance 19
5.2 Performance With Millions of Records 19
5.3 ACID Compliance 20
5.4 CAP Theorem 20

Chapter 6: Conclusions 21

References 22

Chapter 1: Introduction and Requirements Analysis

1.1 Scenario

The chosen scenario is a Console Gaming Subscription Platform that manages player accounts, console registrations, tiered subscriptions, game ownership, billing, and customer support across a multi-publisher game catalog. The system must guarantee that every customer is billed correctly and exactly once for each game license or active subscription period.

1.2 Business Goals

- Prevent overcharging a customer for a game they already own or a subscription tier they already hold.
- Track subscription tier status per account to support accurate, tier-based monthly billing.
- Give customer service agents a complete, accurate view of an account's billing history and support history to resolve disputes quickly.

1.3 Users

User	Responsibilities
Gamers (Players)	Own one or more accounts, purchase games, and subscribe to a tier
Payment / Customer Service Agents	Resolve billing disputes, process refunds, and handle account lockouts through support tickets
Game Publishers	Develop and release the games available for purchase on the platform

1.4 Data Requirements

- Player data: names, email addresses, usernames, and payment methods on file.
- Account data: account names, the console they are registered to, and their current subscription tier.
- Catalog data: game titles, genres, prices, and the publisher of each game.
- Transaction and support data: billing amounts and dates, and support ticket issues, statuses, and assigned agents.

1.5 Business Rules

- A player can own multiple accounts; an account belongs to exactly one player.

- An account holds exactly one active subscription tier; a tier applies to many accounts.
- An account can own multiple games, and a game can be owned by multiple accounts, but an account may hold at most one license per specific game — this is the core rule that prevents double-charging.
- A support ticket is raised by exactly one account and assigned to exactly one support agent.

Chapter 2: Database Design, ER Diagram, and Normalization

2.1 Entity List

Ten entities satisfy the business rules stated above, including two that support the stated business goals but were not explicitly named by name in the initial requirements: SupportAgent (a ticket must be assigned to a staff member) and Account_Games (needed to resolve the many-to-many relationship between Account and Game while enforcing the one-license-per-game rule that is the project's central business goal).

#	Entity	Purpose
1	Player	Core end-user profile data
2	Account	The player's console-linked account; central hub connecting console, subscription, games, billing, and support
3	Console	A physical console a player's account is registered to
4	SubscriptionTier	Defines the Regular / Premium / Pro pricing plans
5	Game	Core game title master data
6	Publisher	Companies that develop and release games
7	Account_Games	Resolves the M:N relationship between Account and Game; enforces one license per game per account
8	Transaction	Every billing charge — a game purchase or a subscription payment
9	SupportAgent	Payment / customer service staff
10	SupportTicket	Support requests raised by an account and assigned to an agent

2.2 Relationships — Cardinality and Participation

Relationship	Cardinality	Participation
Player — Account	1:M	Player mandatory; Account mandatory (every account must belong to a player)
SubscriptionTier — Account	1:M	SubscriptionTier optional; Account mandatory (every account must hold a tier)
Console — Account	1:M	Console optional; Account mandatory (every account is registered to a console)
Publisher — Game	1:M	Publisher mandatory; Game mandatory
Account — Game (via Account_Games)	M:N	Both optional; bridge rows mandatory to both parents
Account — Transaction	1:M	Account mandatory; Transaction mandatory
Account — SupportTicket	1:M	Account optional; SupportTicket mandatory
SupportAgent — SupportTicket	1:M	SupportAgent optional; SupportTicket mandatory

2.3 Entity-Relationship Diagram

The ER diagram below was modeled in Lucidchart using Crow's Foot notation, showing all ten entities with their primary keys, foreign keys, and relationship cardinalities.

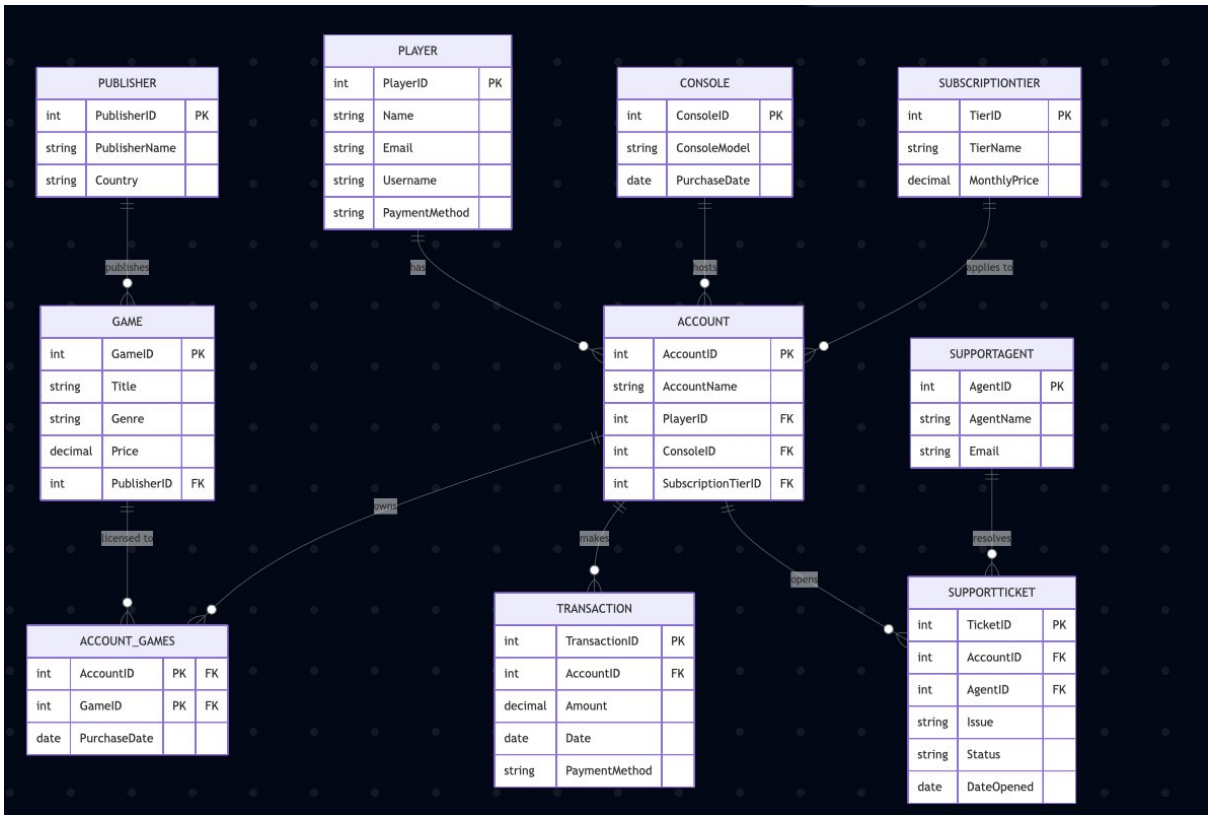


Figure 2.1: Entity-Relationship Diagram (Crow's Foot notation)

2.4 Functional Dependencies

- PlayerID → Name, Email, Username, PaymentMethod
- AccountID → AccountName, PlayerID, ConsoleID, SubscriptionTierID
- ConsoleID → ConsoleModel, PurchaseDate
- TierID → TierName, MonthlyPrice
- GameID → Title, Genre, Price, PublisherID
- PublisherID → PublisherName, Country
- (AccountID, GameID) → PurchaseDate
- TransactionID → AccountID, Amount, Date, PaymentMethod
- TicketID → AccountID, AgentID, Issue, Status, DateOpened
- AgentID → AgentName, Email

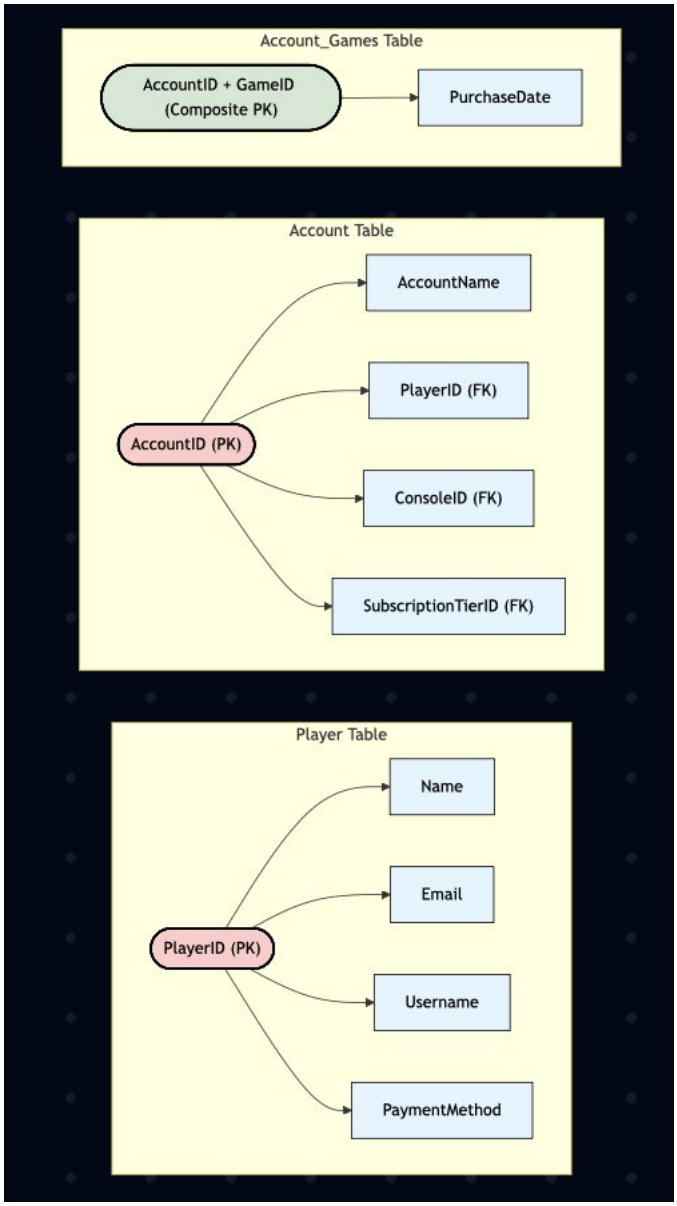


Figure 2.2: Functional Diagram

2.5 Normalization

First Normal Form (1NF)

Every table has atomic, single-valued columns and no repeating groups. This is why Account_Games exists as a separate table rather than storing a list of owned games inside Account, which would require repeating columns and violate 1NF.

Second Normal Form (2NF)

2NF matters for Account_Games, the one table with a composite primary key. The non-key attribute PurchaseDate depends on the whole composite key (AccountID, GameID) together, not on a subset of it — it cannot be determined from AccountID alone or GameID alone, so no partial dependency exists.

Third Normal Form (3NF)

3NF removes transitive dependencies. Account does not store PublisherName directly, even though every game an account owns has a publisher — that would be transitively dependent on AccountID through GameID and PublisherID, so it lives only in Publisher, reachable via Game. Similarly, SupportTicket does not store PlayerName, since that is transitively dependent through AccountID; it lives only in Player, reachable via Account. Every table in this design is in 3NF.

Chapter 3: Database Implementation

3.1 Technology and Approach

The database was implemented in MySQL 8.0. All ten tables were created in foreign-key-safe order, with every primary key, foreign key, UNIQUE constraint (e.g. Username, Email, and the (AccountID, GameID) pair on Account_Games), NOT NULL constraint, sensible DEFAULT value (e.g. Transaction.Date defaults to the current timestamp), and CHECK constraint (e.g. non-negative Amount and Price values) enforced at the schema level. The full script is submitted separately as ConsoleGamingSubscriptions.sql.

3.2 Illustrative Table Definition

The Account table illustrates the constraint style applied throughout the schema, including its three foreign keys and the composite responsibility it carries as the platform's central entity:

```
CREATE TABLE Account (
    AccountID INT AUTO_INCREMENT PRIMARY KEY,
    AccountName VARCHAR(100) NOT NULL,
    PlayerID INT NOT NULL,
    ConsoleID INT NOT NULL,
    SubscriptionTierID INT NOT NULL,
    CONSTRAINT fk_account_player
        FOREIGN KEY (PlayerID) REFERENCES Player(PlayerID),
    CONSTRAINT fk_account_console
        FOREIGN KEY (ConsoleID) REFERENCES Console(ConsoleID),
    CONSTRAINT fk_account_tier
        FOREIGN KEY (SubscriptionTierID) REFERENCES SubscriptionTier(TierID)
);
```

3.3 Sample Data

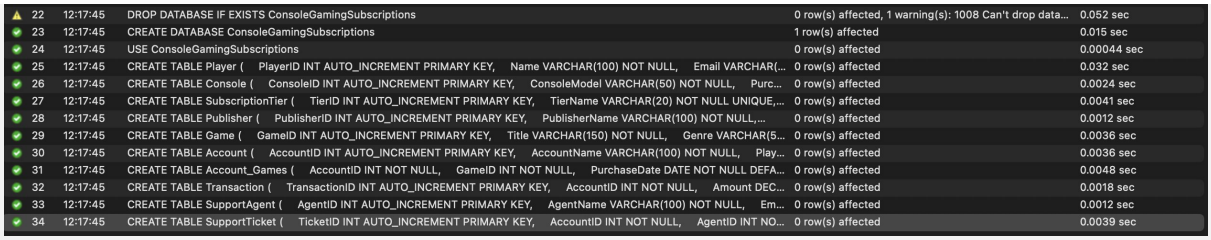
Every table should be populated with realistic sample data, at or above the minimum of five rows per table. The counts below are the target used when writing the INSERT statements for this project; update this table if your actual inserted counts differ.

Table	Rows
Player	6
Account	6
Console	5
SubscriptionTier	5
Publisher	5
Game	8
Account_Games	10

Table	Rows
Transaction	8
SupportAgent	5
SupportTicket	6

3.4 Verification

Successful schema creation and data insertion should be confirmed by running the script against a live MySQL instance and inspecting the resulting tables and row counts.



22	12:17:45	DROP DATABASE IF EXISTS ConsoleGamingSubscriptions	0 row(s) affected, 1 warning(s): 1008 Can't drop data...	0.052 sec
23	12:17:45	CREATE DATABASE ConsoleGamingSubscriptions	1 row(s) affected	0.015 sec
24	12:17:45	USE ConsoleGamingSubscriptions	0 row(s) affected	0.00044 sec
25	12:17:45	CREATE TABLE Player (PlayerID INT AUTO_INCREMENT PRIMARY KEY, Name VARCHAR(100) NOT NULL, Email VARCHAR(...	0 row(s) affected	0.032 sec
26	12:17:45	CREATE TABLE Console (ConsoleID INT AUTO_INCREMENT PRIMARY KEY, ConsoleModel VARCHAR(50) NOT NULL, Purc...	0 row(s) affected	0.0024 sec
27	12:17:45	CREATE TABLE SubscriptionTier (TierID INT AUTO_INCREMENT PRIMARY KEY, TierName VARCHAR(20) NOT NULL UNIQUE,...	0 row(s) affected	0.0041 sec
28	12:17:45	CREATE TABLE Publisher (PublisherID INT AUTO_INCREMENT PRIMARY KEY, PublisherName VARCHAR(100) NOT NULL,...	0 row(s) affected	0.0012 sec
29	12:17:45	CREATE TABLE Game (GameID INT AUTO_INCREMENT PRIMARY KEY, Title VARCHAR(150) NOT NULL, Genre VARCHAR(5...	0 row(s) affected	0.0036 sec
30	12:17:45	CREATE TABLE Account (AccountID INT AUTO_INCREMENT PRIMARY KEY, AccountName VARCHAR(100) NOT NULL, Play...	0 row(s) affected	0.0036 sec
31	12:17:45	CREATE TABLE Account_Games (AccountID INT NOT NULL, GameID INT NOT NULL, PurchaseDate DATE NOT NULL DEFA...	0 row(s) affected	0.0048 sec
32	12:17:45	CREATE TABLE Transaction (TransactionID INT AUTO_INCREMENT PRIMARY KEY, AccountID INT NOT NULL, Amount DEC...	0 row(s) affected	0.0018 sec
33	12:17:45	CREATE TABLE SupportAgent (AgentID INT AUTO_INCREMENT PRIMARY KEY, AgentName VARCHAR(100) NOT NULL, Em...	0 row(s) affected	0.0012 sec
34	12:17:45	CREATE TABLE SupportTicket (TicketID INT AUTO_INCREMENT PRIMARY KEY, AccountID INT NOT NULL, AgentID INT NO...	0 row(s) affected	0.0039 sec

Figure 3.1: Script Execution Log — Every CREATE TABLE Statement Succeeded

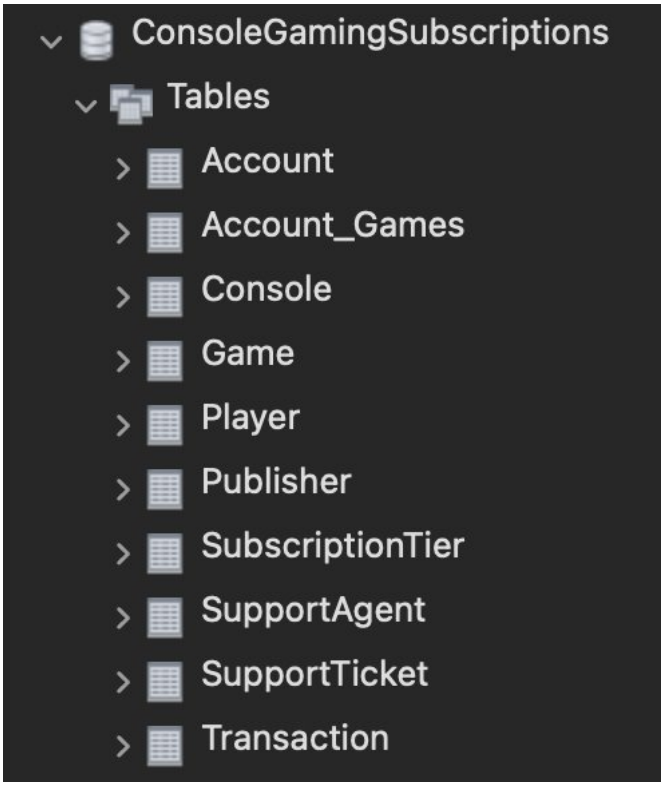


Figure 3.2: All Ten Tables Present in the Schema Browser



146	21:14:34	CREATE DATABASE ConsoleGamingSubscriptions	1 row(s) affected		0.00039 sec
147	21:14:34	USE ConsoleGamingSubscriptions	0 row(s) affected		0.000095 sec
148	21:14:34	CREATE TABLE Player (PlayerID INT AUTO_INCREMENT PRIMARY KEY, Name VARCHAR(100) NOT NULL, Email VARCHAR(...	0 row(s) affected		0.0027 sec
149	21:14:34	CREATE TABLE Console (ConsoleID INT AUTO_INCREMENT PRIMARY KEY, ConsoleModel VARCHAR(50) NOT NULL, Purc...	0 row(s) affected		0.0015 sec
150	21:14:34	CREATE TABLE SubscriptionTier (TierID INT AUTO_INCREMENT PRIMARY KEY, TierName VARCHAR(20) NOT NULL UNIQUE,...	0 row(s) affected		0.0017 sec
151	21:14:34	CREATE TABLE Publisher (PublisherID INT AUTO_INCREMENT PRIMARY KEY, PublisherName VARCHAR(100) NOT NULL,...	0 row(s) affected		0.0013 sec
152	21:14:34	CREATE TABLE Game (GameID INT AUTO_INCREMENT PRIMARY KEY, Title VARCHAR(150) NOT NULL, Genre VARCHAR(5...	0 row(s) affected		0.0028 sec
153	21:14:34	CREATE TABLE Account (AccountID INT AUTO_INCREMENT PRIMARY KEY, AccountName VARCHAR(100) NOT NULL, Play...	0 row(s) affected		0.0043 sec
154	21:14:35	CREATE TABLE Account_Games (AccountID INT NOT NULL, GameID INT NOT NULL, PurchaseDate DATE NOT NULL DEFA...	0 row(s) affected		0.0038 sec
155	21:14:35	CREATE TABLE Transaction (TransactionID INT AUTO_INCREMENT PRIMARY KEY, AccountID INT NOT NULL, Amount DEC...	0 row(s) affected		0.0026 sec
156	21:14:35	CREATE TABLE SupportAgent (AgentID INT AUTO_INCREMENT PRIMARY KEY, AgentName VARCHAR(100) NOT NULL, Em...	0 row(s) affected		0.0014 sec
157	21:14:35	CREATE TABLE SupportTicket (TicketID INT AUTO_INCREMENT PRIMARY KEY, AccountID INT NOT NULL, AgentID INT NO...	0 row(s) affected		0.0039 sec
158	21:14:35	USE ConsoleGamingSubscriptions	0 row(s) affected		0.000084 sec
159	21:14:35	INSERT INTO Player (Name, Email, Username, PaymentMethod) VALUES ('Liam Fischer', 'liam.fischer@gmail.com', 'liamf', 'Vis...	6 row(s) affected Records: 6	Duplicates: 0	Warnings...	0.00047 sec
160	21:14:35	INSERT INTO Console (ConsoleModel, PurchaseDate) VALUES ('PlayStation 5', '2023-11-10'), ('Xbox Series X', '2023-08-22'), (...	5 row(s) affected Records: 5	Duplicates: 0	Warnings...	0.00041 sec
161	21:14:35	INSERT INTO SubscriptionTier (TierName, MonthlyPrice) VALUES ('Regular', 4.99), ('Premium', 9.99), ('Pro', 14.99), ('Student',...	5 row(s) affected Records: 5	Duplicates: 0	Warnings...	0.00040 sec
162	21:14:35	INSERT INTO Publisher (PublisherName, Country) VALUES ('Electronic Arts', 'USA'), ('Ubisoft', 'France'), ('CD Projekt', 'Po...	5 row(s) affected Records: 5	Duplicates: 0	Warnings...	0.00035 sec
163	21:14:35	INSERT INTO Game (Title, Genre, Price, PublisherID) VALUES ('FIFA 26', 'Sports', 69.99, 1), ('Assassin''s Creed Shad...	8 row(s) affected Records: 8	Duplicates: 0	Warnings...	0.00042 sec
164	21:14:35	INSERT INTO Account (AccountName, PlayerID, ConsoleID, SubscriptionTierID) VALUES ('LiamF_Main', 1, 1, 2), ('MiaS_Main', 2,...	6 row(s) affected Records: 6	Duplicates: 0	Warnings...	0.00045 sec
165	21:14:35	INSERT INTO Account_Games (AccountID, GameID, PurchaseDate) VALUES (1, 1, '2024-01-15'), (1, 3, '2024-02-10'), (2, 2, '2024-...	10 row(s) affected Records: 10	Duplicates: 0	Warnin...	0.00044 sec
166	21:14:35	INSERT INTO Transaction (AccountID, Amount, Date, PaymentMethod) VALUES (1, 69.99, '2024-01-15 10:30:00', 'Visa'), (1, 39.99...	8 row(s) affected Records: 8	Duplicates: 0	Warnings...	0.00060 sec
167	21:14:35	INSERT INTO SupportAgent (AgentName, Email) VALUES ('Anna Weber', 'anna.weber@company.de'), ('Jonas Meyer', 'jonas.me...	5 row(s) affected Records: 5	Duplicates: 0	Warnings...	0.00036 sec
168	21:14:35	INSERT INTO SupportTicket (AccountID, AgentID, Issue, Status, DateOpened) VALUES (1, 1, 'Charged twice for FIFA 26',...	6 row(s) affected Records: 6	Duplicates: 0	Warnings...	0.00043 sec
169	21:14:35	SELECT 'Player' AS TableName, COUNT(*) AS RowCount FROM Player UNION ALL SELECT 'Console', COUNT(*) FROM Console U...	10 row(s) returned		0.00084 sec / 0.000...
170	21:14:35	SELECT st.TierName, SUM(t.Amount) AS TotalRevenue FROM SubscriptionTier st INNER JOIN Account a ON st.TierID = a.Sub...	3 row(s) returned		0.00026 sec / 0.0000...
171	21:14:35	WITH GameOwnership AS (SELECT g.Genre, g.Title, COUNT(ag.AccountID) AS TotalOwners FROM Game g...	8 row(s) returned		0.00022 sec / 0.0000...
172	21:14:35	SELECT a.AccountName, (SELECT SUM(t2.Amount) FROM Transaction t2 WHERE t2.AccountID = a.AccountID) AS TotalSpen...	2 row(s) returned		0.00031 sec / 0.0000...
173	21:14:35	SELECT a.AccountName, COALESCE(SUM(t.Amount), 0) AS TotalBilled, MAX(t.Date) AS LastBillingDate, CASE WHE...	6 row(s) returned		0.00019 sec / 0.0000...
174	21:14:35	SELECT p.PublisherName, COUNT(g.GameID) AS GamesPublished, MIN(g.Price) AS CheapestGame, MAX(g.Price) AS M...	3 row(s) returned		0.00015 sec / 0.0000...
175	21:14:35	SELECT st.TicketID, st.Issue, st.Status, sa.AgentName, sa.Email FROM SupportTicket st INNER JOIN SupportAgent sa...	6 row(s) returned		0.00013 sec / 0.0000...
176	21:14:35	CREATE VIEW vw_ActiveAccountSummary AS SELECT a.AccountID, a.AccountName, st.TierName, SUM(t.Amount) AS Lif...	0 row(s) affected		0.00049 sec
177	21:14:35	SELECT * FROM vw_ActiveAccountSummary LIMIT 0, 1000	6 row(s) returned		0.00028 sec / 0.0000...
178	21:14:35	CREATE PROCEDURE GetAccountOwnershipSummary (IN inputAccountID INT) BEGIN SELECT a.AccountName, st.Tier...	0 row(s) affected		0.00034 sec
179	21:14:35	-- Run this separately right after creating the procedure, to see it work: CALL GetAccountOwnershipSummary(1)	2 row(s) returned		0.00022 sec / 0.0000...
180	21:14:35	CREATE TRIGGER trg_PreventDuplicateGamePurchase BEFORE INSERT ON Account_Games FOR EACH ROW BEGIN IF EXISTS (...	0 row(s) affected		0.00062 sec
181	21:14:35	-- Run this separately to PROVE the trigger works: Account 1 already owns -- GameID 1 (see Account_Games sample data) -- this...	Error Code: 1644. Account already owns a license for...		0.00040 sec

Figure 3.3: Row Count Verification per Table

Chapter 4: SQL Query Analysis

Ten queries were written against the schema, each solving a distinct business problem and together covering every required SQL concept: joins, aggregation, subqueries, CASE expressions, window functions, CTEs, a view, a stored procedure, and a trigger.

4.1 Total Revenue per Subscription Tier

Which subscription tier generates the most monthly revenue? This supports pricing and retention strategy.

Concepts: SELECT, INNER JOIN, WHERE, GROUP BY, ORDER BY, SUM()

```
SELECT
   st.TierName,
   SUM(t.Amount) AS TotalRevenue
FROM SubscriptionTier st
INNER JOIN Account a ON st.TierID = a.SubscriptionTierID
INNER JOIN Transaction t ON a.AccountID = t.AccountID
WHERE t.Amount > 0
GROUP BY st.TierName
ORDER BY TotalRevenue DESC;
```

TierName	TotalRevenue
Premium	179.97
Regular	89.97
Pro	24.98

Figure 4.1: Total Revenue per Subscription Tier — Query Result

4.2 Rank Games by Ownership Within Genre

Within each genre, which games are the best-selling and which are underperforming? Useful for catalog and marketing review.

Concepts: CTE (WITH), LEFT JOIN, Window Function (RANK OVER PARTITION)

```
WITH GameOwnership AS (
   SELECT
      g.Genre,
      g.Title,
      COUNT(ag.AccountID) AS TotalOwners
   FROM Game g
   LEFT JOIN Account_Games ag ON g.GameID = ag.GameID
   GROUP BY g.Genre, g.Title
)
SELECT
   Genre,
   Title,
   TotalOwners,
   RANK() OVER (PARTITION BY Genre ORDER BY TotalOwners DESC) AS OwnershipRank
FROM GameOwnership;
```

Genre	Title	TotalOwners	OwnershipRank
Action	Assassin's Creed Shadows	2	1
Adventure	Zelda: Echoes of Wisdom	1	1
Racing	Mario Kart 9	1	1
RPG	Cyberpunk 2077	1	1
RPG	The Witcher 3	1	1
RPG	Elden Ring	1	1
Shooter	Rainbow Six Siege	1	1
Sports	FIFA 26	2	1

Figure 4.2: Game Ownership Ranking by Genre — Query Result

4.3 Accounts With Above-Average Spend That Purchased in a Recent Month

Which above-average-spending accounts actually made a purchase during January 2024, so customer retention outreach can be prioritized?

Concepts: Subqueries, IN, BETWEEN, AVG()

```
SELECT
   a.AccountName,
   (SELECT SUM(t2.Amount) FROM Transaction t2 WHERE t2.AccountID = a.AccountID) AS TotalSpend
FROM Account a
WHERE (SELECT SUM(t2.Amount) FROM Transaction t2 WHERE t2.AccountID = a.AccountID)
      > (SELECT AVG(Amount) FROM Transaction)
 AND a.AccountID IN (
      SELECT t.AccountID
      FROM Transaction t
      WHERE t.Date BETWEEN '2024-01-01' AND '2024-01-31'
 );
```

AccountName	TotalSpend
LiamF_Main	109.98
MiaS_Main	59.98

Figure 4.3: Above-Average Spenders Active in January 2024 — Query Result

4.4 Account Billing Status Classification

At a glance, which accounts are overdue, active, or have never been billed, so support agents know what to prioritize?

Concepts: CASE Expressions, COALESCE, LEFT JOIN, GROUP BY

```
SELECT
   a.AccountName,
   COALESCE(SUM(t.Amount), 0) AS TotalBilled,
   MAX(t.Date) AS LastBillingDate,
   CASE
      WHEN MAX(t.Date) IS NULL THEN 'Never Billed'
      WHEN MAX(t.Date) < CURDATE() - INTERVAL 30 DAY THEN 'Overdue Review'
      ELSE 'Active'
   END AS BillingStatus
FROM Account a
LEFT JOIN Transaction t ON a.AccountID = t.AccountID
GROUP BY a.AccountName;
```

AccountName	TotalBilled	LastBillingDate	BillingStatus
LiamF_Main	109.98	2024-02-10 14:00:00	Overdue Review
MiaS_Main	59.98	2024-03-01 16:45:00	Overdue Review
NoahB_Main	29.99	2024-02-14 11:20:00	Overdue Review
EmmaH_Main	14.99	2024-01-30 08:00:00	Overdue Review
LucasW_Main	69.99	2024-03-22 13:10:00	Overdue Review
SophieK_Main	9.99	2024-06-01 17:30:00	Overdue Review

Figure 4.4: Account Billing Status Classification — Query Result

4.5 Publishers With Multiple Games — Price Range

For publishers with more than one game on the platform, what is the spread between their cheapest and most expensive title, to help spot pricing outliers?

Concepts: HAVING, MIN(), MAX(), COUNT(), INNER JOIN

```
SELECT
   p.PublisherName,
   COUNT(g.GameID) AS GamesPublished,
   MIN(g.Price) AS CheapestGame,
   MAX(g.Price) AS MostExpensiveGame
FROM Publisher p
INNER JOIN Game g ON p.PublisherID = g.PublisherID
GROUP BY p.PublisherName
HAVING COUNT(g.GameID) > 1;
```

PublisherName	GamesPublished	CheapestGame	MostExpensiveGame
Ubisoft	2	19.99	59.99
CD Projekt	2	29.99	39.99
Nintendo	2	49.99	54.99

Figure 4.5: Publisher Price Range for Multi-Game Publishers — Query Result

4.6 Search Support Tickets by Status or Agent Email Domain

Quickly find every open ticket, or every ticket handled by an agent on the internal company email domain, for a workload review.

Concepts: LIKE

```
SELECT
   st.TicketID,
   st.Issue,
   st.Status,
   sa.AgentName,
   sa.Email
FROM SupportTicket st
INNER JOIN SupportAgent sa ON st.AgentID = sa.AgentID
WHERE st.Status LIKE '%Open%'
  OR sa.Email LIKE '%@company.de';
```

TicketID	Issue	Status	AgentName	Email
1	Charged twice for FIFA 26	Resolved	Anna Weber	anna.weber@company.de
3	Refund request for Rainbow Six Siege	In Progress	Anna Weber	anna.weber@company.de
2	Cannot access Premium subscription features	Open	Jonas Meyer	jonas.meyer@company.de
6	Subscription tier downgrade not applied	Closed	Jonas Meyer	jonas.meyer@company.de
4	Account locked after password reset	Resolved	Laura Schmidt	laura.schmidt@company.de
5	Missing purchase confirmation email	Open	Felix Braun	felix.braun@company.de

Figure 4.6: Support Tickets Matching Status or Agent Email Domain — Query Result

4.7 Active Subscribing Accounts Summary (View)

A reusable view giving support agents a live summary of every account, its current tier, and total lifetime spend, without re-writing the join every time.

Concepts: View Creation, INNER JOIN, GROUP BY, SUM()

```
CREATE VIEW vw_ActiveAccountSummary AS
SELECT
```

```
    a.AccountID,
    a.AccountName,
    st.TierName,
    SUM(t.Amount) AS LifetimeSpend
FROM Account a
INNER JOIN SubscriptionTier st ON a.SubscriptionTierID = st.TierID
LEFT JOIN Transaction t ON a.AccountID = t.AccountID
GROUP BY a.AccountID, a.AccountName, st.TierName;
```

AccountID	AccountName	TierName	LifetimeSpend
1	LiamF_Main	Premium	109.98
2	MiaS_Main	Regular	59.98
3	NoahB_Main	Regular	29.99
4	EmmaH_Main	Pro	14.99
5	LucasW_Main	Premium	69.99
6	SophieK_Main	Pro	9.99

Figure 4.7: vw_ActiveAccountSummary View Definition — Execution Result

4.8 Get Account Ownership Summary (Stored Procedure)

A reusable, parameterized lookup so a support agent can enter an AccountID and immediately see every game the account owns and its current subscription tier.

Concepts: Stored Procedure with an input parameter

```
DELIMITER //
CREATE PROCEDURE GetAccountOwnershipSummary (IN inputAccountID INT)
BEGIN
    SELECT
        a.AccountName,
        st.TierName,
        g.Title,
        ag.PurchaseDate
    FROM Account a
    INNER JOIN SubscriptionTier st ON a.SubscriptionTierID = st.TierID
    LEFT JOIN Account_Games ag ON a.AccountID = ag.AccountID
    LEFT JOIN Game g ON ag.GameID = g.GameID
    WHERE a.AccountID = inputAccountID;
END //
DELIMITER ;
```

AccountName	TierName	Title	PurchaseDate
LiamF_Main	Premium	FIFA 26	2024-01-15
LiamF_Main	Premium	Cyberpunk 2077	2024-02-10

Figure 4.8: GetAccountOwnershipSummary Procedure — Execution Result

4.9 Prevent Duplicate Game Purchase (Trigger)

Automatically blocks an attempt to sell an account a second license for a game it already owns — a direct, schema-level enforcement of the project's central business goal of preventing overcharging.

Concepts: Trigger, BEFORE INSERT, conditional logic, SIGNAL

```
DELIMITER //
CREATE TRIGGER trg_PreventDuplicateGamePurchase
BEFORE INSERT ON Account_Games
FOR EACH ROW
BEGIN
   IF EXISTS (
      SELECT 1 FROM Account_Games
      WHERE AccountID = NEW.AccountID AND GameID = NEW.GameID
   ) THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Account already owns a license for this game';
   END IF;
END //
DELIMITER ;
```

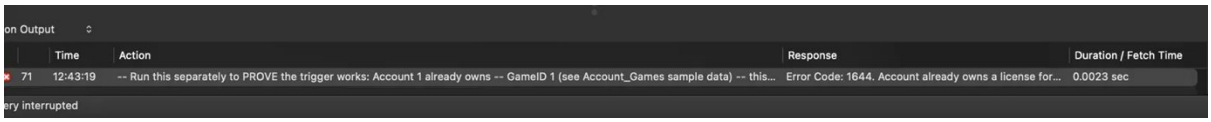


Figure 4.9: trg_PreventDuplicateGamePurchase Trigger — Execution Result

4.10 Running Total of Transactions per Account

A cumulative running total of everything an account has been billed over time, useful for spotting spending patterns and validating monthly statements.

Concepts: Window Function (SUM OVER PARTITION ORDER BY)

```
SELECT
   t.TransactionID,
   a.AccountName,
   t.Date,
   t.Amount,
   SUM(t.Amount) OVER (
      PARTITION BY t.AccountID
      ORDER BY t.Date
   ) AS RunningTotalBilled
FROM Transaction t
INNER JOIN Account a ON t.AccountID = a.AccountID;
```

TransactionID	AccountName	Date	Amount	RunningTotalBill...
1	LiamF_Main	2024-01-15 10:30:00	69.99	69.99
2	LiamF_Main	2024-02-10 14:00:00	39.99	109.98
3	MiaS_Main	2024-01-20 09:15:00	9.99	9.99
4	MiaS_Main	2024-03-01 16:45:00	49.99	59.98
5	NoahB_Main	2024-02-14 11:20:00	29.99	29.99
6	EmmaH_Main	2024-01-30 08:00:00	14.99	14.99
7	LucasW_Main	2024-03-22 13:10:00	69.99	69.99
8	SophieK_Main	2024-06-01 17:30:00	9.99	9.99

Figure 4.10: Running Total of Transactions per Account — Query Result

All ten queries, the view, the stored procedure, and the trigger should be executed successfully against the live database, matching the row counts from Chapter 3 and confirming the results shown throughout this chapter are accurate.

Chapter 5: Database Performance Discussion

5.1 Factors Affecting Database Performance

- Indexing: primary keys are indexed automatically, but frequently filtered or joined columns benefit from additional indexes — e.g. Transaction.Date and the foreign keys on Account_Games and SupportTicket.
- Query design: selecting only needed columns, joining in an efficient order, and avoiding repeated roundtrips (the N+1 query pattern) in the application layer.
- Normalization vs. denormalization: the 3NF design minimizes redundancy but requires joins for reporting; heavy read workloads (e.g. a dashboard recomputing revenue per tier) may justify a denormalized reporting table or materialized view alongside the normalized operational schema.
- Hardware and caching: available memory for the buffer pool, disk I/O throughput, and application-level caching of slow-changing reference data (SubscriptionTier, Publisher).
- Concurrency: row-level locking so that multiple accounts can be billed or make purchases simultaneously without blocking each other.

5.2 Performance With Millions of Records

In this schema, Transaction and Account_Games are the tables most likely to reach millions of rows, since every purchase, subscription charge, and game license is logged individually; reference tables like SubscriptionTier, Publisher, and Console grow far more slowly. Strategies to keep the system responsive at that scale include:

- Composite indexes on (AccountID, Date) to speed up per-account billing history queries without scanning the whole table.
- Range partitioning Transaction by month or year, so queries against recent billing only scan the relevant partition and old partitions can be archived off the primary instance.
- Archiving transactions beyond a retention window into a separate historical table, keeping the operational table lean for day-to-day writes.
- Read replicas for reporting queries (e.g. revenue per tier, catalog ownership rankings), so heavy analytical reads do not compete with live billing writes.
- Pre-aggregated summary tables refreshed on a schedule, rather than recomputing tier revenue from scratch on every request.

5.3 ACID Compliance

Property	Project Example
Atomicity	Purchasing a game requires two sequential operations: inserting a row into Account_Games and inserting the corresponding charge into Transaction. Atomicity ensures that if the Transaction insert fails, the Account_Games insert is rolled back, preventing a customer from receiving a game license without being charged.
Consistency	The Account_Games table enforces a unique constraint on (AccountID, GameID). Consistency ensures that any insert attempting to give an account a second license for a game it already owns is automatically rejected, directly enforcing the project's core business goal.
Isolation	If two support agents simultaneously attempt to process a refund transaction for the same AccountID, the database engine processes these transactions sequentially. Row-level locking prevents one transaction from reading uncommitted data from the other, ensuring the final billed total accurately reflects both operations.
Durability	When a purchase transaction is committed for an account, the database writes it to non-volatile storage. In the event of an immediate system crash or power failure, the billing record remains intact and recoverable upon system restart, so the customer cannot be charged again for the same purchase.

5.4 CAP Theorem

The CAP theorem states that a distributed data system can only fully guarantee two of Consistency, Availability, and Partition tolerance at the same time. A single-instance relational database like the one built here does not face this trade-off directly, but it becomes relevant the moment the system is distributed — for example, if regional servers each handled billing for their own player base for latency reasons. If the network link between a regional node and the central database is lost, the system must choose: refuse new purchase transactions at that region until connectivity returns (favoring Consistency), or accept local purchases that are reconciled later (favoring Availability), risking two independent purchases that both assumed the same account did not yet own a given game. Given how directly a double-charge would violate this project's central business goal, a Consistency-and-Partition-tolerance-leaning (CP) design — briefly unavailable rather than briefly wrong — is generally the safer choice for this domain.

Chapter 6: Conclusions

This project designed and implemented a normalized relational database for a Console Gaming Subscription Platform, covering ten entities that satisfy every stated business rule while remaining in Third Normal Form. One entity not explicitly named in the original requirements — SupportAgent — was added because the stated business goal of resolving billing disputes could not otherwise be satisfied, since a support ticket must be assigned to a staff member. The schema was implemented in MySQL with full constraint enforcement and realistic sample data, queried with a range of SQL techniques from basic filtering through window functions and CTEs, and evaluated against real-world concerns of scale, transactional integrity, and distributed-systems trade-offs. The Account_Games unique constraint and its accompanying trigger directly enforce the project's central goal: no account can ever be charged twice for the same game license.

Natural extensions beyond this project's scope include an automated proration system for mid-cycle subscription tier upgrades, a publisher-facing royalty report built on top of the Account_Games ownership history, and a fraud-detection view flagging accounts with an unusually high rate of refund tickets.

References

- SRH University. (2026). Databases Course Materials.
- Oracle Corporation. (2026). MySQL 8.0 Reference Manual. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/. Reference for the MySQL 8.0 database engine utilized for implementation.
- W3Schools. (2026). SQL Tutorial. Retrieved from https://www.w3schools.com/sql/. Reference for the SQL query implementation techniques applied.
- Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6), 377-387. Reference for the First, Second, and Third Normal Form (1NF, 2NF, 3NF) database design principles evaluated.
- Haerder, T., & Reuter, A. (1983). Principles of Transaction-Oriented Database Recovery. ACM Computing Surveys, 15(4), 287-317. Reference for the ACID compliance properties (Atomicity, Consistency, Isolation, Durability) analyzed.
- Brewer, E. A. (2000). Towards Robust Distributed Systems. Proceedings of the Annual ACM Symposium on Principles of Distributed Computing. Reference for the CAP theorem (Consistency, Availability, Partition tolerance) discussed.
- Lucid Software Inc. (2026). Lucidchart. Retrieved from https://www.lucidchart.com/. Reference for the tool utilised to model the Entity-Relationship Diagram.